

Python Basics Documentation

Official Reference Guide to Core Syntax, Control Flow, and Data Structures

1. Python Language Overview & Syntax

Python is a high-level, interpreted, general-purpose programming language known for clean syntax and readability. Python code executes sequentially via the CPython interpreter, compiling source code (.py) into bytecode (.pyc) before execution.

```
def calculate_stats(numbers: list[float]) -> dict[str, float]:
    """Calculate basic statistical metrics from a list of numbers."""
    total = sum(numbers)
    count = len(numbers)
    mean = total / count if count > 0 else 0.0
    return {"total": total, "count": count, "mean": mean}

data = [10.5, 20.0, 30.5, 40.0]
result = calculate_stats(data)
print(f"Stats Result: {result}")
```

2. Control Flow Structures

Python supports standard control flow mechanisms including conditional branches (if/elif/else), iteration loops (for/while), list comprehensions, and structural pattern matching (match/case).

```
status_code = 404
match status_code:
    case 200:
        print("Success Request")
    case 404:
        print("Resource Not Found")
    case 500 | 502 | 503:
        print("Server Error")
    case _:
        print("Unknown Status")
```

3. Data Structures & Memory Management

Python built-in containers include Lists (mutable arrays), Tuples (immutable arrays), Dictionaries (hash maps with O(1) lookup), and Sets (unique element collections). Python uses automatic reference counting and a generational garbage collector.

```
# Dictionary & Set Operations
user_roles = {"alice": "admin", "bob": "editor", "charlie": "viewer"}
active_users = {"alice", "bob"}

# Filter active admins
active_admins = {u for u in active_users if user_roles.get(u) == "admin"}
print("Active Admins:", active_admins)
```